

ARC-AGI-3: Independent Strategy Audit and Experimental Solver Report

Research and engineering report

September 16, 2026

Abstract

ARC-AGI-3 rewards discovering how unfamiliar interactive environments work and solving them with few actions. The central recommendation is selective, revisable modeling with decision-focused experiments, not blind action-sequence enumeration or indiscriminate verification. An independent symbolic prototype was implemented and evaluated during the earlier work recorded in this conversation. It performs well on deliberately narrow synthetic diagnostic tasks, but has not been tested on official public environments and cannot be described as competition-ready. Faraday was not supplied, so no source-based or empirical head-to-head comparison was possible.

Delivery note. This is a reconstructed report copy. When the user requested a direct download, the current workspace contained only the original blank template PDF; the earlier report, solver source and experiment artifacts were no longer available there. This document restores the substantive analysis and recorded results from the conversation. Its experiment figures refer to the earlier tool executions, not newly rerun experiments. It is not a byte-for-byte restoration of the earlier 16-page PDF.

Contents

1	Executive judgment	3
2	What the benchmark asks	3
2.1	Observations and actions	3
2.2	Levels, persistence and hidden evaluation	4
2.3	Scoring and action efficiency	4
2.4	What was actually inspected	5
3	What strong public approaches establish	5

4	Challenge the candidate approaches	6
5	Compute allocation and human-like solving	7
6	Ideal architecture and its failure tests	8
7	Independent prototype implemented earlier	9
8	Recorded experiments and iteration	9
8.1	Protocol	9
8.2	What changed because of evidence	10
9	Faraday comparison and outstanding requirements	11
10	Final recommendation and roadmap	12

1 Executive judgment

The fundamental problem is action-costly program identification and control under partial observability. The agent must discover meaningful state, controls, rules and goals while paying for interaction. Planning becomes useful when a predictive abstraction exists, but a search algorithm does not itself supply the right abstraction or objective.

There is no established single universal bottleneck. Missing information, inadequate representations, wrong goals, poor experiments, proposal failures and execution errors can each dominate different games. Goal induction is a plausible frequently observed failure in the supplied blind-public runs, not a structural invariant of the benchmark.

The strongest general strategy should combine broad priors with falsifiable predictions, targeted intervention, persistent history, and short, conditionally trusted plans. External executable models can be valuable, especially for long-horizon composition. They are an engineering choice to test against an equally capable persistent reasoning agent, not an empirically proven necessity.

Important evidence boundaries:

- Official facts, published results, supplied project claims and locally executed experiments are different evidence classes.
- Public success does not establish transfer to intentionally different private mechanics.
- A high score from a powerful hosted model is not a demonstrated recipe for a small offline agent.
- The independent prototype’s synthetic scores are not ARC scores or estimates of hidden performance.
- Without Faraday’s files, no claim that the new solver outperforms Faraday is justified.

2 What the benchmark asks

2.1 Observations and actions

The inspected interface supplies ordered frames, status, completed-level count, total win-level count, action metadata and available actions. Multiple frames can result from one action. Discarding intermediate animation frames can discard evidence. The visual representation uses integer color indices on small grids, with dimensions up to 64×64 and 16 colors.[2, 3]

ACTION1–4 are conventionally directional; ACTION5 is conventionally interaction; ACTION6 is a coordinate click; ACTION7 is conventionally undo. These conventions are not instructions revealing a particular game’s mechanics. RESET must be treated according to the actual environment wrapper and evaluator; it is not a free branching simulator.[3]

The documented deployment-facing interface supports observing and acting, not privileged recovery of hidden source or free engine clones. An offline debugging capability must not be assumed to exist

behind the competition scoring gateway. The inspected starter uses an agent subclass with a method receiving frame history and the latest frame.[5]

2.2 Levels, persistence and hidden evaluation

The technical report describes 135 selected environments: 25 public, 55 semi-private for API-model evaluation, and 55 fully private for competition. Thus the combined 110 nonpublic environments are not one interchangeable Kaggle evaluation pool. Their mechanics deliberately differ from public material.[1]

Within a game, levels can share mechanics while changing layouts or teaching additional dependencies. Carry rules and their evidence, not old coordinates or walkability maps. A level-completion animation can already contain the next level’s initial state, so do not learn that visual jump as an ordinary action effect.

Keep three kinds of persistence separate: engine state, agent memory within a game, and memory across games. The actual runner and storage policy determine cross-environment persistence. Frozen weights and code are different from mutable test-time memory. The starter’s continuing object lifetime alone does not establish unrestricted cross-environment learning.[5]

Determinism given a seed and full engine state does not imply that the current image is Markov. History, counters and latent phases can change outcomes from the same visible state. The appropriate general object is a belief over programs and hidden state conditional on the observation/action history. Identifiability is relative to the interaction policy and available budget.

2.3 Scoring and action efficiency

The current methodology inspected during the earlier research uses a squared action-efficiency ratio and linear level-index weights. For positive human reference counts and nonzero agent action counts, write

$$e_{g\ell} = c_{g\ell} \min \left(1.15, (h_{g\ell}/a_{g\ell})^2 \right),$$

where $c_{g\ell}$ indicates completion. The game score is the weighted efficiency, capped at the weighted completed-level share:

$$S_g = 100 \min \left(\frac{\sum_{\ell=1}^{L_g} \ell e_{g\ell}}{\sum_{\ell=1}^{L_g} \ell}, \frac{\sum_{\ell=1}^{L_g} \ell c_{g\ell}}{\sum_{\ell=1}^{L_g} \ell} \right).$$

Environment scores are averaged. The methodology describes an upper-median successful-human reference, while the March report describes a second-best-human reference and a different per-level cap. These differences require pinning the operative evaluator rather than treating one release description as timeless.[4, 1]

Without a cap binding, twice the reference actions gives one quarter of a level’s efficiency; five times gives one twenty-fifth. This is arithmetic, not a proof against every search strategy. Early experimentation can still be valuable when it saves actions across later levels.

The March report discusses a five-times-human cutoff per level; the supplied project describes gateway-enforced budgets. The actual current gateway was unavailable for verifying the precise

scope. The local synthetic harness instead used a clearly labeled fixed call budget. Internal reasoning is not directly charged by the action metric, but hardware and wall limits still matter.[1, 6]

2.4 What was actually inspected

Published pixels from VC33, LS20 and FT09 were inspected in Figure 2 of the independent *Graph-Based Exploration* paper. VC33’s vertical structures, LS20’s irregular layout and FT09’s colored cell array suggest different grouping and control hypotheses. A universal avatar assumption is unjustified. A screenshot does not establish collision, toggle or goal semantics.[7]

No public game was played in the available runtime. The official SDK, assets, weights and API key were absent; terminal network requests were blocked. Browser-based literature and image inspection is not equivalent to environment interaction.

3 What strong public approaches establish

The following are published results discussed in the earlier research, not replications by this prototype. Comparisons across model capability, settings and access conditions are not controlled experiments.

Retrodict reports 99.86% public mean RHAЕ and all 183 levels with GPT-5.6 Sol at max reasoning, with approximately \$654 API-list-price cost. Its historical-rule checking and detailed logs support the viability of the modeling route, not its necessity for all agents.[8]

Twin reports 93.3% versus 61.1% for an off-the-shelf coding-agent ablation, clearing 179/183 levels. Multiple mechanisms change in that ablation. The reported first-seen/recurring prediction split of 79.0%/94.8% warns that familiar-transition agreement can overstate frontier reliability. Truly latent state is a stated limitation.[9]

Tycho compares direct reasoning, actor-controlled model building and automatic falsification-triggered model building in public runs. The discussed scores are 79.07, 88.49 and 83.07 respectively. More accurate reconstructed transitions did not automatically produce the highest game score. This favors selective checking rather than blanket replay gating, with single-run limitations retained.[10]

NVIDIA AVO reports 100 public RHAЕ and 183/183 levels with Opus 5, using exact textual grids, persistent memory and supervision. It follows direct interaction rather than making an ARC-specific executable-model layer central. This does not rule out internal modeling; it does counter an established-necessity claim for an external version space. Its comparison to other systems is not a controlled ablation.[11]

Astra has a reported 99.95% semi-private Provider Adapter/high result versus 62.71% Standard/max. The adapter changes opaque reasoning-state persistence and context handling, not merely the size of ordinary text memory. Different harnesses and inference settings matter. Observed symbolic modeling supports the importance of useful internal representations, not a directly accessible local-offline implementation.[12]

Offline-agent precedents in the official milestone report include Duck’s local coding loop, Reki’s

local VLM emitting short action batches, and forge’s configurable framework. Forge’s best profile disabled its extra machinery. These motivate a capable lightweight baseline rather than assuming every handcrafted subsystem helps.[13]

StochasticGoose’s earlier 12.58% preview result used 255,964 actions in a different regime. The preview had three private environments; the table’s two games denotes completions. It is an existence proof for non-LLM agents, not evidence about efficiency under the present constraints.[14]

4 Challenge the candidate approaches

Blind action-sequence enumeration. It needs an action alphabet and success signal but little semantic information. At branching factor b and depth d , up to b^d sequences may be considered, with actual execution and reset costs. Sparse feedback barely prunes the space. It avoids wrong-model failures by having no model, but long horizons and click coordinates are severe problems. A tiny last-action lottery can be rational; global sequence guessing is a poor default.

Heuristic search. It solves goal-directed reachability given a state, successor function and useful heuristic. Internal search can save real actions; physical successor exploration pays for each branch. A wrong goal or progress proxy can prioritize irrelevant states. Transfer comes from the learned abstraction and heuristic, not the priority queue. Long horizons need subgoals or structure.

Monte Carlo tree search. It allocates planning effort among branches but needs a simulator or costly interaction. Sparse wins make uninformed rollouts weak; long horizons increase variance. A wrong learned model can make simulated evidence confidently misleading. Use MCTS when uncertainty and branching justify it, not simply because the benchmark is interactive.

Model-based planning. It composes transitions toward a proposed goal. It requires state sufficient for prediction and a model credible on the reachable plan frontier. Sparse rewards remain a goal-discovery problem. A single critical dependency error can break an otherwise accurate plan. Receding-horizon execution and targeted frontier checks reduce, but do not eliminate, this risk.

Executable world models. They permit simulation, replay and counterexample repair. A runnable program is not a verified causal model: it can memorize a log, omit latent state or fail only near the goal. Compute can substitute for interaction when the program captures real mechanisms. Code generation requires a trustworthy observation contract and sandboxing.

Symbolic or state-machine induction. It offers compact rules and explicit consistency checks within a language. It requires suitable entities, predicates and latent constructs. The grammar can become enormous, while novel mechanics outside it are not solvable by a better proposal distribution inside that same grammar. Empty version spaces must remain explicit failures.

Direct LLM reasoning. It provides broad visual, causal and goal priors without a predetermined DSL. Quality depends on model capability, observation fidelity, context management and memory. It can handle sparse feedback using priors but can also hallucinate rules. Latency and tokens replace some action cost. Selective generation versus generation every action should be measured, not prohibited by doctrine.

LLM plus external verifier. This separates broad proposal from consistency checking. Replay rejects known contradictions but cannot certify future correctness. An incomplete verifier can validate a misleading abstraction. The interesting control is an equally persistent reasoner with identical budgets, not deliberately memoryless prompting.

LLM plus learned policy or value model. It amortizes deliberation and can guide planning, but requires suitable training tasks and targets. Public-game overfitting, miscalibrated values and misleading reward proxies threaten transfer. A confident value function can discard initially useless-looking but necessary probes. Learn and deploy these components after validating the behavior being amortized.

Memorization and caches. They are excellent for validated exact repeats and cheap to execute. False matches waste actions; storing more public solutions is not the same as discovering private mechanics. Keep verified caches as separately measured fast paths, not evidence of novel-game reasoning.

Hybrid systems. Cheap perception and execution plus expensive reasoning at uncertainty boundaries is attractive. But modules need explicit ownership of beliefs and failure signals; otherwise one module treats another’s guess as truth. Hybridization itself does not provide transfer. Add a subsystem only when its marginal value survives a fair comparison.

5 Compute allocation and human-like solving

With unlimited internal reasoning and charged interaction, compute should reduce decision-relevant uncertainty: compare representations, recover events, propose rules and goals, find reachable separating experiments, and check consequential plan steps. No amount of thought distinguishes hypotheses that remain observationally equivalent under every affordable policy.

A useful conceptual objective is expected future score improvement plus decision-focused value of information, minus risk. Information gain alone is insufficient: learning everything about a decoration may be less useful than testing one goal-relevant dependency.

Human-like solving can be decomposed into computational primitives:

1. Notice repetition, symmetry, salient changes and possible controls; preserve alternative groupings.
2. Choose a low-risk intervention that separates competing interpretations.

3. Compare before, during and after the intervention, tracking events and identity.
4. Change a relevant factor and test again instead of accepting correlation as causation.
5. Predict an unseen consequence before observing it.
6. Distinguish visual progress proxies from actual completion events.
7. Transfer rules and their evidence across levels, while rechecking exceptions and controls.

This is an analytical decomposition, not a new human-subject study. Humans also overfit and miss latent variables.

6 Ideal architecture and its failure tests

The recommended system has six responsibilities: lossless observation memory; revisable state; predictive and goal alternatives; decision-focused probes; conditional planning; and execution with prediction checks and budget accounting. These need not be six large modules.

Consistency is not confidence. Surviving replay means only consistency with known evidence. A probe should allow informative surprise. A commit deserves stronger immediate-frontier support and should stop on contradiction. Concentrate expensive checking where an error changes the decision, rather than prohibiting exploration.

Failure challenge	Required response and unresolved limit
Entirely new mechanic	Propose a new representation or program, not only new parameters inside familiar templates. A fixed DSL cannot guarantee this.
Wrong first hypothesis	Preserve alternatives, test separating predictions, invalidate plans after contradiction.
Deceptive feedback	Keep actual completion separate from visual progress; maintain alternative goals.
Hidden variable	Retain history and consider bounded counters or automata. Detecting aliasing is not uniquely recovering hidden state.
Initially useless action	Consider delayed enabling effects and information value; immediate visual change is not the only measure.
Levels teach different rules	Carry supported knowledge with provenance; do not assume the first level identifies every dependency.
Huge search space	Use learned proposals and decision-focused discrimination rather than exhaustive program enumeration.
One critical model error	Check the reachable frontier and execute short prefixes. Another replay pass is still not a completeness proof.
Public/private mismatch	Keep public evaluation as regression; reserve hidden-generalization claims for external hidden evidence.

The revised architecture is selective, uncertainty-aware modeling, not an obligation to build a perfect simulator before taking an action. A partial model can support a sound local choice without justifying a long unsupported commitment.

7 Independent prototype implemented earlier

The earlier implementation used Python’s standard library for production code. It included connected same-color components, fixed categorical sampling sites, factored action-rule alternatives, goal priors, model-state BFS, disagreement probes, conditional carry, and JSONL logs preserving all intermediate frames.

The rule family included identity, constants, copies, binary cycles and one equality guard. Rules contradicted by recorded transitions were removed. Empty factors remained unknown. Planning expanded induced states, not hidden-engine clones, and execution replanned after every real action.

Goal priors were deliberately narrow: all sites equal or all sites one color. A proposed sufficient goal seen without completion was rejected. A completion could support a model-predicted terminal predicate, explicitly labeled conditional evidence rather than a directly observed universal rule.

After a same-state/different-outcome contradiction, the second version tried an action-count parity feature and retained it only when current-level history became replay-consistent. This was one bounded history hypothesis, not general hidden-state recovery.

Limitations. Fixed sites do not model arbitrary object motion or changing grouping. Stored subframes were not semantically interpreted. The implementation had no capable LLM, general program synthesis, training, structural-dependency metric, evaluator-owned lockbox or expressibility oracle. SDK and starter bridges were tested only against stubs. A real SDK invocation failed because the packages and assets were unavailable.

The previously generated solver source is not present in the current delivery workspace. This reconstructed PDF preserves its design and recorded evidence, but should not be mistaken for a downloadable restoration of the complete code project.

8 Recorded experiments and iteration

8.1 Protocol

All worlds were inspectable, self-authored DEV tasks, never public ARC games or lockboxes. Six families were used: independent toggles, coupled toggles, conditional enabling, remapped controls, a latent counter and a changing distracting HUD. They shared a stable click-board representation, a substantial selection bias.

Each comparison used three seeds, six families and three levels per game: 54 possible level completions per method. The cap was 80 calls per environment, including the initial reset. Search limits were 256 nodes per search and depth eight, with a common 20-second cooperative wall limit. No headline run hit the wall limit. There was no GPU or model-token budget because no model was used.

Exact shortest solutions were computed by evaluator-only full-state BFS before agent execution. The agent never received those solutions or the environment’s rule functions. Reference lengths were not human baselines. Therefore the following are raw progress and call counts, not RHAЕ.

Setting	Model	No history	Random	Enumeration
Four sites: levels /54	54	48	43	11
Four sites: total calls	466	571	1065	1440
Four sites: total seconds	0.68	0.66	1.26	1.23
Six sites: levels /54	50	49	23	12
Six sites: total calls	860	899	1349	1440
Six sites: total seconds	3.31	3.24	4.61	3.28

Table 2: Results recorded in the earlier execution. Four-site seeds were 17, 29 and 43; six-site seeds were 101, 211 and 307. Timings are container measurements, not hardware-normalized benchmarks.

Family	Four sites, /9			Six sites, /9		
	Model	Random	Enum.	Model	Random	Enum.
Independent	9	7	3	9	3	2
Coupled	9	9	1	9	6	3
Conditional	9	5	3	9	3	2
Remapped	9	5	1	9	5	2
Latent counter	9	9	0	5	4	1
Distracting HUD	9	8	3	9	2	2

Table 3: The larger counter cases expose a continuing failure. No-history obtained 3/9 and 4/9 on the respective counter families and matched Model elsewhere.

8.2 What changed because of evidence

Version 1 solved 48/54 levels. On the four-site counter family it solved only 3/9, while random solved 9/9. Traces showed identical encoded states with different outcomes and empty rule factors. This implicated the state abstraction, not simply insufficient action-tree search.

The evidence-triggered parity feature improved those counter results to 9/9, using 135 calls across three runs instead of 240. Scaling was less impressive: six-site counters improved only from 4/9 to 5/9. On seed 101 the history proposal arrived at charged action 66; on seed 211 it was never accepted. Experiment selection and delayed refinement therefore remain real limitations.

A separately labeled post-failure diagnostic increased both variants’ caps to 400 calls, 4,096 nodes and depth 16 on six-site counter seed 101. The history model solved 3/3 in 93 calls; no-history solved 1/3 after 400. Because several caps changed together, this does not isolate which resource increase helped. The original failure remains in the headline results.

Prediction accuracy did not track progress monotonically. On four-site DEV, the model’s available predictions were correct on 144/307 transitions; random’s passive learner was correct on 620/797. They visited different states with different coverage and recurrence, so these are not causal cross-policy accuracy comparisons. Choosing useful novel actions can reduce apparent on-policy predictive accuracy.

Enumeration and random shared scene extraction and passive learning for measurement. Enumeration already received component-center clicks rather than all pixels, but its lexicographic reset-and-replay schedule was not optimized. The results support model-directed action selection on these

families, not a theorem against enumeration and not an answer to the persistent-LLM decisive question.

An accidentally launched prepatch run compared identical action selectors. It was explicitly labeled non-admissible and excluded. The earlier final validation recorded **28 passing tests**, covering rule falsification, aliases, planning, subframes, accounting, boundaries, reference solutions, score arithmetic and interface stubs. The final coupled smoke case completed three levels in 11 charged calls against reference lengths of 2, 4 and 2.

Zero public episodes were executed. That is not the same claim as running public episodes and solving zero levels. Nor do the synthetic successes establish competition readiness.

9 Faraday comparison and outstanding requirements

Faraday was not found in the supplied workspace after the independent work was frozen. No source, checkpoint, cache, harness or logs were inspected. The comparison is therefore between demonstrated prototype capabilities and explicitly attributed claims in the user’s plan, not an empirical head-to-head.

Aspect	Independent prototype	Faraday, as described by the supplied plan
Deployment maturity	DEV runtime and stub adapters; official run blocked.	Shipped harness and Kaggle packaging. If confirmed, materially more mature.
Perception	Static per-color sampling sites.	HUD deception handling, motion extraction and kinematics. Claimed coverage is broader.
Modeling evidence	Reproducible narrow rule/history intervention in the earlier runs.	Science loop and proposed version-space/generator work; actual status unverified.
Goals	Equal/color priors with conditional evidence.	Designer-prior lattice and opportunistic goal banking. Richer claimed priors are not proof of hidden transfer.
Budgets	Fixed calls and cooperative wall checks.	Claimed official scoring, enforced budgets and wall caps. Preserve that infrastructure if it verifies.
Hidden performance	None measured.	Claims or pending results in the plan, not reproduced. No winner can be declared.

Faraday is described as the broader and more mature engineering system. The independent prototype demonstrates a compact mechanism on limited diagnostic tasks, but cannot claim superiority over Faraday even there without running both on the same tasks. It should not replace verified deployment infrastructure on the strength of synthetic percentages.

To complete the requested comparison, supply Faraday and an already installed official runtime with assets. Freeze both agents, match observations, action accounting, seeds, hardware and wall envelopes, then compare progress, efficiency and trace-based failure attribution. The required real public evaluation and source comparison remain unfinished.

10 Final recommendation and roadmap

1. **Start with trustworthy execution.** Obtain official assets and validate the gateway, observations, level transitions and accounting before interpreting architecture results.
2. **Use a capable persistent reasoning control.** Match perception, memory, model capability and compute. Do not substitute random or memoryless prompting for that decisive comparison.
3. **Add explicit model machinery selectively.** Test whether proposal, replay and disagreement management save enough actions to justify their compute and complexity.
4. **Repair the measured failure.** Missing state calls for history/perception work; poor separation calls for probes; known expressibility with poor proposals calls for better proposal distributions; language limits require widening the representation.
5. **Keep DEV, fresh lockboxes and hidden results separate.** Adaptive consultation consumes a holdout even without reading its source. Report all freezes and attempts.
6. **Train after mechanisms validate.** Compare success-only imitation against revision/probe training on DEV, then confirm a frozen choice once on fresh evaluation.

The strongest recommendation is not “search harder” or “always build a perfect simulator.” It is to spend computation on a predictive abstraction and on experiments that change the next important decision. Search belongs where that abstraction makes it informative. Do not spend the initial budget on blind deep enumeration, privileged hidden-engine search, public-game scripts advertised as generality, or unverified large rollouts.

For an expressibility pivot, name a frozen resource-bounded hypothesis class and distinguish language failure from missing information, proposal failure and execution failure. A failed state-oracle run alone is insufficient. Exhaustive equivalence claims require a genuinely finite, covered state/history space. Ground-truth rule oracles must be evaluator-owned and checked beyond the recorded log. None of those certificates was implemented in this prototype.

Bottom line: the earlier work produced useful research evidence and a small end-to-end diagnostic agent, not a demonstrated competitive ARC-AGI-3 solver. The decisive next evidence is actual public execution and a matched persistent-reasoner comparison, followed by properly separated hidden evaluation.

References

- [1] ARC Prize, *ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence*, arXiv:2603.24621v1. <https://arxiv.org/html/2603.24621v1>.

- [2] ARC Prize, *Game Schema*. <https://docs.arcprize.org/game-schema>.
- [3] ARC Prize, *Actions*. <https://docs.arcprize.org/actions>.
- [4] ARC Prize, *Scoring Methodology*. <https://docs.arcprize.org/methodology>.
- [5] ARC Prize, Kaggle starter agent. https://github.com/arcprize/ARC-AGI-3-Kaggle-Start/blob/main/agent/my_agent.py.
- [6] ARC Prize, competition documentation. <https://docs.arcprize.org/arc-prize-2026>.
- [7] *Graph-Based Exploration for ARC-AGI-3 Interactive Reasoning Tasks*, arXiv:2512.24156. <https://arxiv.org/pdf/2512.24156>.
- [8] Ryan Brown, *Retrodict*. <https://github.com/ryanbbrown/Retrodict>.
- [9] Skoutnev et al., *Twin: Playing an Unknown Game with a Test-Time Digital Twin*, arXiv:2608.14490v1. <https://arxiv.org/html/2608.14490v1>.
- [10] *Tycho: Active Abstraction with Programmatic World Models for ARC-AGI-3*. <https://github.com/NIMI-research/Tycho>.
- [11] NVIDIA, *NVIDIA AVO Reaches 100% on ARC-AGI-3*. <https://developer.nvidia.com/blog/nvidia-avo-reaches-100-on-arc-agi-3-demonstrating-a-frontier-level-general-purpose-architecture-for-long-horizon-autonomous-agents/>.
- [12] ARC Prize, *Astra* and detailed results, September 2026. <https://arcprize.org/blog/astra>; <https://arcprize.org/results/openai-gpt-6-astra>.
- [13] ARC Prize, *ARC Prize 2026: Milestone Prize #1*, July 6, 2026. <https://arcprize.org/blog/arc-prize-2026-milestone-1>.
- [14] ARC Prize, *ARC-AGI-3 Preview: 30-Day Learnings*. <https://arcprize.org/blog/arc-agi-3-preview-30-day-learnings>.